Python代码-06-计算成本/更新矩阵

1. 只能焊接碳钢的焊工550/天；只能焊接不锈钢的焊工600/天；碳钢和不锈钢均可以焊接的焊工650/天；
2. 共现矩阵x轴下方增加每天焊工数、工作量、剩余量、成本的计算和输出。

遗传算法 复制代码

```
import pandas as pd
import numpy as np
from deap import base, creator, tools, algorithms
import random
from sqlalchemy import create_engine
from collections import defaultdict

# MySQL 连接信息
mysql_connection_string =
'mysql+pymysql://ncc_01:ncc%21%40%23%24@211.138.201.6:3506/pcms_2022'
mysql_engine = create_engine(mysql_connection_string)

# 查询语句
query = """
SELECT *
FROM v_pp_weldlist
WHERE unitCode = '马来西亚项目'
AND (alterType IS NULL OR alterType NOT IN ('c', 'f', 'w'))
AND TRIM(remark) <> ''
"""
# 执行查询
df_mysql = pd.read_sql(query, mysql_engine)

# 关闭数据库连接
mysql_engine.dispose()

# 导出到 Excel
excel_path = '数据表.xlsx'
df_mysql.to_excel(excel_path, index=False)

# 使用pandas的isna()函数来检查空值
df = df_mysql[pd.isna(df_mysql['finishDate']) |
(df_mysql['finishDate'].astype(str).str.strip() == '')].copy()

# 读取 Excel 文件
```

```python
df_welders = pd.read_excel('焊工台账.xlsx')

# 筛选 '焊接对象' 为 '管道' 并且 '焊工证下发情况' 为 '已下发'
df_welders_filtered = df_welders[(df_welders['焊接对象'] == '管道') &
(df_welders['焊工证下发情况'] == '已下发')]

# 提取 '焊工号' 和 'WPS No.'
df_welders_filtered = df_welders_filtered[['焊工号', 'WPS No.']]

# 读取 WPS 数据
df_wps = pd.read_excel('WPS.xlsx')

# 定义材料映射字典
material_mapping = {
    'A105': 'A106 Gr.B',
    'A106': 'A106 Gr.B',
    'A106-CE': 'A106 Gr.B',
    'API 5L': 'A106 Gr.B',
    'A53 Gr.B': 'A106 Gr.B',
    'A335 GR.P11': 'P11'
}

# 对于不在映射字典中的材料，保持原值
def map_material(material):
    if material in material_mapping:
        return material_mapping[material]
    else:
        return material

# 将df中的材料名称映射到WPS中使用的标准名称
df['material_standard'] = df['material'].apply(map_material)

# 解析壁厚范围，创建新列
df_wps[['min_thickness', 'max_thickness']] = df_wps['壁厚范围'].str.split('~',
expand=True).astype(float)

# 初始化匹配的WPS列表列
df['matched_WPS'] = np.empty((len(df), 0)).tolist()

# 遍历每行数据，匹配WPS
for index, row in df.iterrows():
    matched_wps = df_wps[(df_wps['材质'] == row['material_standard']) &
                         (df_wps['min_thickness'] <= row['thickness']) &
                         (df_wps['max_thickness'] >= row['thickness'])]['PQR
No.'].tolist()
```

```
    df.at[index, 'matched_WPS'] = matched_wps

    # 如果没有找到匹配的WPS编号，打印警告
    if not matched_wps:
        print(f"警告：行 {index} 没有匹配到任何WPS编号。材料：
{row['material_standard']}，厚度：{row['thickness']}")

# 关联df_welders_filtered和df_wps来获取材质分类
df_welders_materials = df_welders_filtered.merge(df_wps[['PQR No.', '材质分
类']], left_on='WPS No.', right_on='PQR No.', how='left')

# 去重，确保每个焊工对每个PQR No.只保留一条记录
df_welders_materials = df_welders_materials.drop_duplicates(subset=['焊工号',
'PQR No.'])

# 按焊工号分组，获取每个焊工的材质分类列表
welder_materials = df_welders_materials.groupby('焊工号')['材质分
类'].apply(set).reset_index()

# 计算每个焊工的日成本
def calculate_daily_cost(materials):
    if materials == {'碳钢'}:
        return 550
    elif materials == {'不锈钢'}:
        return 600
    elif materials == {'碳钢', '不锈钢'}:
        return 650
    else:
        return 550  # 默认成本，如果材质分类为空或其他情况

welder_materials['日成本'] = welder_materials['材质分
类'].apply(calculate_daily_cost)

# 合并回df_welders_filtered以包含日成本
df_welders_filtered = df_welders_filtered.merge(welder_materials[['焊工号', '日
成本']], on='焊工号', how='left')

# 创建适应度函数和个体类
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))  # 最小化适应度函数
creator.create("Individual", list, fitness=creator.FitnessMin)  # 基于适应度函数
的个体类

# 创建一个字典来存储每个WPS可以由哪些焊工执行
wps_to_welders = defaultdict(set)
for _, row in df_welders_filtered.iterrows():
```

```
    for wps in [row['WPS No.']]:  # 确保 WPS No. 正确处理
        wps_to_welders[wps].add(row['焊工号'])

# 将集合转换为列表（如果需要在之后的处理中使用列表）
for wps in wps_to_welders:
    wps_to_welders[wps] = list(wps_to_welders[wps])

# 改进的初始化策略
def create_individual():
    individual = []
    for _, task in df.iterrows():
        # 获取这个任务对应的合格焊工列表
        qualified_welders = set()
        for wps in task['matched_WPS']:
            if wps in wps_to_welders:
                qualified_welders.update(wps_to_welders[wps])

        qualified_welders = list(qualified_welders)  # 将集合转换为列表以使用
random.choice

        if qualified_welders:
            welder = random.choice(qualified_welders)
        else:
            # 如果没有合格焊工，分配默认值
            welder = "NONE"
            print(f"No qualified welders found for WPS: {task['matched_WPS']}")

        individual.append(welder)
    return creator.Individual(individual)

# 设置遗传算法的工具箱
toolbox = base.Toolbox()
toolbox.register("attr_welder", random.choice, df_welders_filtered['焊工
号'].unique().tolist())  # 随机选择一个焊工号作为属性
toolbox.register("individual", create_individual)  # 使用新的个体创建策略
toolbox.register("population", tools.initRepeat, list, toolbox.individual)  #
创建种群，种群由多个个体组成

# 构建管线的应检测点口数量映射
line_to_inspection_spots = {}
for line, group in df_mysql.groupby('lineCode'):
    if 'rtRate' in group.columns and not group['rtRate'].isnull().all():
        rt_rate = group['rtRate'].dropna().astype(float).iloc[0] / 100  # 取第
一个非空的rtRate并转换为比例
        if rt_rate > 0:  # 只有当rtRate大于0时，计算应检测点口数量
```

```
            bw_count = group[group['jtype'] == 'BW'].shape[0]
            if bw_count > 0:  # 只有当存在BW类型的焊接时，才需要检测
                inspection_spots = np.ceil(bw_count * rt_rate)
                line_to_inspection_spots[line] = int(inspection_spots)
    else:
        line_to_inspection_spots[line] = 0  # 若无rtRate或全部为NaN，设置检测点口
数量为0

# 构建管线到已用焊工的映射
line_welders = defaultdict(set)
for line, welder in df_mysql[['lineCode', 'coverWelder']].dropna().values:
    line_welders[line].add(welder)

# 评估函数
def evalWelding(individual):
    total_days = 0
    total_penalty = 0
    total_cost = 0  # 总成本初始化
    tasks_remaining = df.copy()
    task_days = {}  # 记录任务被安排的天数
    daily_summaries = []  # 新增一个列表来存储每天的摘要信息
    task_details = {}  # 新建字典来记录每个任务的详细信息

    # 初始化welders_per_line包含line_welders中的数据
    welders_per_line = defaultdict(set, {k: set(v) for k, v in
line_welders.items()})

    while not tasks_remaining.empty:
        daily_cost = 0  # 每天的成本初始化
        daily_workloads = {}
        tasks_to_drop = []
        welders_used_today = set() # 记录当天使用的焊工

        for i, (index, task) in enumerate(tasks_remaining.iterrows()):
            welder = individual[i]
            line_code = task['lineCode']
            # 获取该管线的应检测点口数量
            inspection_spots = line_to_inspection_spots.get(line_code, 0)

            if welder == "NONE":
                tasks_to_drop.append(index)
                continue

            if welder not in daily_workloads:
                daily_workloads[welder] = 0
```

```
            if daily_workloads[welder] + task['jsize'] <= 50:
                daily_workloads[welder] += task['jsize']
                tasks_to_drop.append(index)
                task_days[index] = total_days + 1  # 记录任务的安排天数
                welders_used_today.add(welder)
                task_details[index] = {'welder': welder, 'day': total_days +
1}  # 记录任务的详细信息
            else:
                continue

            # 计算超出的焊工号数
            if inspection_spots > 0 and welder != "NONE":
                # 在添加前检查焊工号是否已在集合中
                initial_welder_count = len(welders_per_line[line_code])

                # 添加焊工号
                welders_per_line[line_code].add(welder)

                # 如果集合大小增加了，说明这是一个新的焊工号
                if len(welders_per_line[line_code]) > initial_welder_count:
                    # 如果超出应检测点口数量，增加惩罚
                    if len(welders_per_line[line_code]) > inspection_spots:
                        total_penalty += 10

        daily_total = sum(daily_workloads.values())
        daily_cost = sum(df_welders_filtered.loc[df_welders_filtered['焊工号']
== welder, '日成本'].iloc[0] for welder in welders_used_today)  # 计算当天的总成
本
        tasks_remaining.drop(tasks_to_drop, inplace=True)
        daily_summaries.append(f"第 {total_days + 1} 天: 完成 {daily_total} 寸,
剩余总寸径量 {tasks_remaining['jsize'].sum()}, 当天焊工人数
{len(welders_used_today)}, 当天焊工成本 {daily_cost}")

        # 惩罚1: 每天焊接量不足1500寸径量
        if daily_total < 1500:
            total_penalty += 100  # 不足1500寸径量的惩罚

        # 惩罚2: 每天每有一个焊工没有安排工作
        total_penalty += (len(df_welders_filtered['焊工号'].unique()) -
len(welders_used_today)) * 5

        # 惩罚3: 每个派出的焊工每天工作量不足20寸径量
        for welder, workload in daily_workloads.items():
            if workload < 20:
```

南京南化建设有限公司 夏斐 | 5299

```
                total_penalty += 5  # 每个工作不饱和的焊工惩罚

        total_cost += daily_cost  # 累加每天的成本到总成本
        total_days += 1
        print(f"第 {total_days} 天: 完成 {daily_total} 寸, 剩余总寸径量 {tasks_remaining['jsize'].sum()}, 当天焊工人数 {len(welders_used_today)}, 当天焊工成本 {daily_cost}")

        if not tasks_to_drop:  # 如果一天没有任务被分配
            total_penalty += 1000  # 增加高额惩罚以避免无限循环
            break

    print(f"个体评估: 总天数 = {total_days}, 总惩罚值 = {total_penalty}, 总焊工成本 = {total_cost}\n")  # 打印每个个体的总天数和总惩罚值和总焊工成本
    return total_days, total_penalty, task_days, daily_summaries, total_cost, task_details  # 返回总天数和任务天数映射 # 返回每个任务的详细信息

# 注册遗传算法的操作
toolbox.register("evaluate", evalWelding)  # 注册评估函数，确保它只返回适应度相关的一个值
toolbox.register("mate", tools.cxTwoPoint)  # 注册交叉操作
toolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.2)  # 注册变异操作
toolbox.register("select", tools.selTournament, tournsize=3)  # 注册选择操作

# 运行遗传算法
population = toolbox.population(n=15)  # 初始化种群
NGEN = 12  # 设置迭代次数
for gen in range(NGEN):
    print(f"第 {gen + 1} 代: ")
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.8, mutpb=0.2)  # 生成后代
    # 使用map函数来计算每个个体的适应度
    fits = list(toolbox.map(toolbox.evaluate, offspring))
    # 更新适应度
    for fit, ind in zip(fits, offspring):
        # 增加总天数的权重因子
        weighted_fitness = 10000 * fit[0] + 10 * fit[1] + fit[4]  # fit[0] 是总天数，fit[1] 是总惩罚值，fit[4] 是总成本
        ind.fitness.values = (weighted_fitness,)  # 更新适应度值

    population = toolbox.select(offspring, k=len(population))  # 选择新的种群

# 输出最佳解
best_ind = tools.selBest(population, 1)[0]  # 选择最佳个体
```

```
total_days, total_penalty, task_days, best_daily_summaries, total_cost,
best_task_details = evalWelding(best_ind)  # 独立调用评估，获取详细结果

# 输出每日摘要信息
print("最优解的每日任务摘要：")
for summary in best_daily_summaries:
    print(summary)
print(f"个体评估：总天数 = {total_days}，总惩罚值 = {total_penalty}，总焊工成本 =
{total_cost}\n")

# 再次计算最佳个体的加权适应度，以便输出
best_fitness = 10000 * total_days + 10 * total_penalty + total_cost
best_ind.fitness.values = (best_fitness,)  # 更新适应度

# 更新DataFrame
df['Assigned Welder'] = df.index.map(lambda x: best_task_details[x]['welder'])
df['Day Assigned'] = df.index.map(lambda x: best_task_details[x]['day'])

# 输出和保存结果
output_excel_path = 'Updated_Data.xlsx'  # 指定输出Excel文件路径
```

共现矩阵 复制代码

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from mpl_toolkits.axes_grid1.inset_locator import inset_axes

# 设置 matplotlib 支持中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']  # 设置使用的字体
plt.rcParams['axes.unicode_minus'] = False  # 正确显示负号

# 读取数据
df = pd.read_excel('Updated_Data.xlsx')

# 映射 'areaCode' 列的值
area_code_map = {'900-01': '900', '900-02': '900', '900-03': '900', '900-04':
'900', '900-05': '900'}
df['areaCode'] = df['areaCode'].replace(area_code_map)

# 创建一个透视表来计算共现矩阵
co_occurrence_matrix = df.pivot_table(index='areaCode', columns='Day
Assigned', values='Assigned Welder', aggfunc=lambda x: len(set(x)))

# 重命名行和列的名称
```

```python
co_occurrence_matrix.index.name = '单元'
co_occurrence_matrix.columns.name = '天数'

# 计算每天派出的焊工总数量
daily_welders_total = df.groupby('Day Assigned')['Assigned Welder'].nunique()

# 计算当天安排的工作量
daily_workload = df.groupby('Day Assigned')['jsize'].sum()

# 计算剩余总工作量
total_jsize = df['jsize'].sum()
cumulative_workload = daily_workload.cumsum()
remaining_workload = total_jsize - cumulative_workload

# 合并这些信息到一个 DataFrame
summary_df = pd.DataFrame({
    '焊工总数量': daily_welders_total,
    '当天工作量': daily_workload,
    '剩余工作量': remaining_workload
})

# 读取 Excel 文件
df_welders = pd.read_excel('焊工台账.xlsx')

# 筛选 '焊接对象' 为 '管道' 并且 '焊工证下发情况' 为 '已下发'
df_welders_filtered = df_welders[(df_welders['焊接对象'] == '管道') &
(df_welders['焊工证下发情况'] == '已下发')]

# 提取 '焊工号' 和 'WPS No.'
df_welders_filtered = df_welders_filtered[['焊工号', 'WPS No.']]

# 读取 WPS 数据
df_wps = pd.read_excel('WPS.xlsx')

# 关联df_welders_filtered和df_wps来获取材质分类
df_welders_materials = df_welders_filtered.merge(df_wps[['PQR No.', '材质分
类']], left_on='WPS No.', right_on='PQR No.', how='left')

# 去重，确保每个焊工对每个PQR No.只保留一条记录
df_welders_materials = df_welders_materials.drop_duplicates(subset=['焊工号',
'PQR No.'])

# 按焊工号分组，获取每个焊工的材质分类列表
welder_materials = df_welders_materials.groupby('焊工号')['材质分
类'].apply(set).reset_index()
```

```python
# 计算每个焊工的日成本
def calculate_daily_cost(materials):
    if materials == {'碳钢'}:
        return 550
    elif materials == {'不锈钢'}:
        return 600
    elif materials == {'碳钢', '不锈钢'}:
        return 650
    else:
        return 550  # 默认成本，如果材质分类为空或其他情况

welder_materials['日成本'] = welder_materials['材质分
类'].apply(calculate_daily_cost)

# 合并回df_welders_filtered以包含日成本
df_welders_filtered = df_welders_filtered.merge(welder_materials[['焊工号', '日
成本']], on='焊工号', how='left')

# 从df_welders_filtered中提取焊工成本信息
welder_costs = df_welders_filtered[['焊工号', '日成
本']].drop_duplicates().set_index('焊工号')

# 计算每天的焊工总成本，确保每个焊工只计算一次
daily_welder_costs = df.groupby('Day Assigned')['Assigned
Welder'].apply(lambda welders: sum(welder_costs.loc[w, '日成本'] for w in
set(welders) if w in welder_costs.index))

# 更新summary_df DataFrame以包含每天的总成本
summary_df['焊工总成本'] = daily_welder_costs

# 计算成本汇总
cumulative_cost = daily_welder_costs.cumsum()

# 创建图形和主轴
fig, ax = plt.subplots(figsize=(20, 15))

# 绘制热图
sns.heatmap(co_occurrence_matrix, annot=True, cmap='Blues', cbar=True,
linewidths=.5, yticklabels=1, xticklabels=1, ax=ax)
plt.yticks(rotation=0)  # 将纵轴标签设置为水平显示
plt.title('各单元每日派工数量共现矩阵')

# 添加第二个轴用于显示统计信息，位置在主轴下方，但更靠近 X 轴
```

```
ax2 = inset_axes(ax, width="100%", height="5%", loc='lower center',
bbox_to_anchor=(0, -0.05, 1, 1),
                 bbox_transform=ax.transAxes, borderpad=0)
ax2.set_xlim(ax.get_xlim())
ax2.set_xticks(ax.get_xticks())
ax2.set_xticklabels([f'焊工数量: {daily_welders_total.get(day, 0)}人\n工作量:
{daily_workload.get(day, 0)}寸\n剩余量: {remaining_workload.get(day, 0)}寸\n当日
成本: {daily_welder_costs.get(day, 0)}元\n汇总成本: {cumulative_cost.get(day,
0)}元'
                     for day in co_occurrence_matrix.columns], ha='center',
fontsize=7)  # 调小字号
ax2.tick_params(axis='x', rotation=0)  # 取消标签旋转

# 隐藏 ax2 的 Y 轴和边框，因为它只用于显示 X 轴标签
ax2.set_frame_on(False)
ax2.get_yaxis().set_visible(False)

# 保存图片，设置高清晰度
plt.savefig('各单元每日派工数量共现矩阵.png', dpi=300, bbox_inches='tight')

# 显示图片
```

南京南化建设有限公司 夏斐 | 5299